

# Mais itens

Duas poções novas, seguindo o molde de `usar(dono, inimigo)` que vocês já construíram.

## NESTA AULA

1. Recapitular o padrão de item
2. Construir a Poção de força
3. Construir o Anel de proteção
4. Encaixar os dois nas fases

---

### Antes de começar, confira a sua pasta:

|                            |                                                                                                 |
|----------------------------|-------------------------------------------------------------------------------------------------|
| <code>personagem.py</code> | Personagem, <code>_vida</code> , <code>curar</code> , <code>ataque</code> , <code>defesa</code> |
| <code>itens.py</code>      | Item, <code>PocaoDeVida</code> , <code>Bomba</code> , <code>Inventario</code>                   |
| <code>classes.py</code>    | Guerreiro, Mago, Goblin, Orc, Troll, Dragao                                                     |
| <code>main.py</code>       | Fase, a lista <code>fases</code> , o menu, o combate                                            |

## ANTES DE COMEÇAR

# Onde vocês pararam

- Todo item tem `usar(self, dono, inimigo)`. É a promessa da classe `Item`.
- A Poção de vida cura `dono`. A Bomba fere `inimigo`. Mesma chamada, efeitos opostos.
- Cada `Fase` tem um prêmio, e o prêmio é só mais um item guardado na mochila.

```
item = jogador.inventario.tirar(escolha)
item.usar(jogador, inimigo)
```

É essa linha, dentro do `main.py`, que vai chamar os itens novos de hoje — sem precisar mudar nem uma vírgula nela. É por isso que vale a pena revisar o padrão antes de escrever o primeiro item.

**VOCABULÁRIO RÁPIDO, SE ALGUÉM ESQUECEU**

|                           |                                                                                                            |
|---------------------------|------------------------------------------------------------------------------------------------------------|
| <code>Item</code>         | a classe-mãe de todo item. Só promete um <code>usar()</code> , não faz nada sozinha.                       |
| <code>usar()</code>       | o método que todo item sobrescreve para fazer seu próprio efeito.                                          |
| <code>Inventario</code>   | a mochila. Guarda os itens numa lista privada, <code>_itens</code> .                                       |
| <code>Fase</code>         | tem um <code>nome</code> , uma <code>descricao</code> , um <code>inimigo</code> e um <code>premio</code> . |
| <code>polimorfismo</code> | a mesma chamada, <code>item.usar(...)</code> , se comporta diferente conforme o item.                      |

# Poção de força

---

Duas poções só de cura ficam repetitivas. Hoje entra uma poção que não cura — ela deixa o próprio personagem mais forte.

A pergunta de hoje: **o que muda no código, e o que continua igual**, entre uma poção que já existe e uma poção nova?

Todo RPG com inventário faz essa mesma escolha de design: em Zelda existe a Poção Vermelha (cura) e o Elixir Azul (deixa mais forte por um tempo). Em quase todo jogo de turno existe pelo menos um item que cura e outro que ataca ou fortalece. Vocês estão construindo a versão mínima desse padrão.

## O que a Poção de força precisa fazer:

- Guardar um nome e um número — quanto de ataque ela dá.
  - Ter um `usar(self, dono, inimigo)`, como todo item já tem.
  - Aumentar o `ataque` de quem bebeu, e avisar com uma mensagem.
- 

### QUANTO CÓDIGO ISSO É

Oito linhas. É a classe mais curta que vocês vão escrever hoje — e ainda assim ela vai funcionar no jogo inteiro, porque segue o contrato que já existe.

# O que muda, o que fica igual

## Já existe: PocaDeVida

```
class PocaDeVida(Item):  
    def __init__(self):  
        super().__init__(  
            "Poção de vida")  
        self.cura = 25  
  
    def usar(self, dono, inimigo):  
        dono.curar(self.cura)  
        print(dono.nome,  
              "recuperou vida")
```

## Nova: PocaDeForca

```
class PocaDeForca(Item):  
    def __init__(self):  
        super().__init__(  
            "Poção de força")  
        self.forca = 5  
  
    def usar(self, dono, inimigo):  
        dono.ataque += self.forca  
        print(dono.nome,  
              "ganhou ataque")
```

■ muda na PocaDeVida    ■ muda na PocaDeForca

### AS TRÊS DIFERENÇAS, E SÓ ESSAS TRÊS

O nome do item ( "Poção de vida" vira "Poção de força" ), o nome do atributo ( cura vira forca ), e a linha de efeito dentro de usar ( curar() vira ataque += ). O resto — o \_\_init\_\_, a assinatura de usar, a forma do print — é cópia exata.

### SAÍDA AO USAR POCAODEVIDA

Thoric recuperou vida

### SAÍDA AO USAR POCAODEFORCA

Thoric ganhou ataque

# Por que nada mais muda

As duas classes têm exatamente a mesma forma: um `__init__` que guarda um número, e um `usar` que aplica esse número em `dono`.

Só o meio muda — `curar()` vira `ataque +=`. A assinatura `usar(self, dono, inimigo)` é sempre a mesma, e é exatamente por isso que o resto do jogo nunca precisa saber qual item está na mão do personagem.

Isso é polimorfismo de novo — a mesma ideia da Parte B, só que agora entre objetos que não são personagens. `jogador.inventario.tirar(n).usar(jogador, inimigo)` roda sem nenhum `if` perguntando qual item é.

## O PONTO DA AULA

É por isso que a mochila, o menu e a fase não precisam saber qual poção é qual. Todos respondem à mesma chamada: `item.usar(jogador, inimigo)`.

## PERGUNTE À TURMA

E se a força devesse durar só aquela luta, em vez de para sempre? O que precisaria mudar? Não implementem agora — só pensem em voz alta.

Resposta esperada: precisaria de um jeito de "desligar" o bônus no fim da luta — provavelmente guardando o quanto foi somado e descontando de volta, ou usando o `iniciar_turno()` que já existe. Não é óbvio, e é por isso que hoje o bônus fica permanente.

# Anel de proteção

---

Agora um item que aumenta a defesa em vez do ataque. Antes de mostrar o código: pensem sozinhos por um instante em qual linha da `PocaoDeForca` precisa mudar.

A resposta é curta, e é exatamente por isso que vale a pena escrever no quadro antes de virar a página.

Reparem no nome escolhido: **Anel de proteção**, não "anel temporário". O bônus que ele dá fica para sempre — chamar ele de temporário seria um nome que promete algo que o código não faz. Nome errado confunde mais do que nome nenhum.

## ERRO COMUM DE COPIAR E COLAR

Quem copia a `PocaoDeForca` inteira e só troca o nome da classe costuma esquecer de trocar `ataque` por `defesa` dentro do `usar`. O programa não quebra — o anel só continua aumentando o ataque, com um nome que diz outra coisa. Confiram essa linha com atenção.

# Mesma forma, atributo diferente

## Já fizemos: PocaDeForca

```
class PocaDeForca(Item):  
    def __init__(self):  
        super().__init__(  
            "Poção de força")  
        self.forca = 5  
  
    def usar(self, dono, inimigo):  
        dono.ataque += self.forca  
        print(dono.nome,  
              "ganhou ataque")
```

## Nova: AnelDeProtecao

```
class AnelDeProtecao(Item):  
    def __init__(self):  
        super().__init__(  
            "Anel de proteção")  
        self.protecao = 5  
  
    def usar(self, dono, inimigo):  
        dono.defesa += self.protecao  
        print(dono.nome,  
              "ganhou defesa")
```

 muda na PocaDeForca     muda na AnelDeProtecao

### DE NOVO, AS TRÊS DIFERENÇAS

Nome do item, nome do atributo ( `forca` vira `protecao` ), e o atributo do personagem que recebe o bônus ( `ataque` vira `defesa` ). Reconhecer isso de novo, na segunda vez, sozinhos, é o exercício de hoje.

#### SAÍDA AO USAR POCAODEFORCA

Thoric ganhou ataque

#### SAÍDA AO USAR ANELDEPROTECAO

Thoric ganhou defesa

# Três itens, um padrão só

Poção de vida, Poção de força, Anel de proteção. Três classes, três efeitos diferentes, e a mesma forma exata por dentro.

Quando o padrão se repete três vezes sem exceção, ele deixa de ser coincidência — vira a regra de como todo item deste jogo vai ser escrito daqui pra frente.

| ITEM             | ATRIBUTO              | EFEITO EM <small>USAR</small>             |
|------------------|-----------------------|-------------------------------------------|
| Poção de vida    | <code>cura</code>     | <code>dono.curar(self.cura)</code>        |
| Poção de força   | <code>forca</code>    | <code>dono.ataque += self.forca</code>    |
| Anel de proteção | <code>protecao</code> | <code>dono.defesa += self.protecao</code> |

## O PONTO DA AULA

Um item novo nunca pede para mudar `Inventario`, `Fase` ou o menu. Se algum desses arquivos precisou mudar para o item novo funcionar, alguma coisa foi feita fora do padrão.

## PERGUNTE À TURMA

Seguindo esse padrão, que quarto item vocês inventariam? Qual seria o nome, o atributo, e quem ele afeta — `dono` ou `inimigo`?

Exemplos prontos, se a turma travar: uma Bomba de fogo (mais dano que a Bomba comum, afeta `inimigo`), um Elmo (mais defesa, igual ao anel), uma Adaga envenenada (reduz o ataque do `inimigo` em vez de aumentar o do `dono`).



## JUNTANDO COM O QUE JÁ EXISTE

# Os itens viram prêmio de fase

```
fases = [  
    Fase("Floresta Sombria", "Um Goblin salta na sua frente.",  
        Goblin(), PocaDeVida()),  
    Fase("Trilha da Montanha", "Um Orc bloqueia o caminho.",  
        Orc(), PocaDeForca()),  
    Fase("Ponte Quebrada", "Um Troll ronca embaixo da ponte.",  
        Troll(), AnelDeProtecao()),  
    Fase("Covil do Dragão", "O Dragão Ancião desperta.",  
        Dragao(), PocaDeVida()),  
]
```

Nada além dessa lista muda. `fase.premio` já sabia entregar qualquer item — inclusive um que ainda não existia ontem.

## O que acontece quando alguém vence a Trilha da Montanha:

1. `combate(jogador, fase.inimigo)` roda até o Orc perder.
2. `jogador.inventario.adicionar(fase.premio)` guarda a `PocaDeForca()` que foi criada lá em cima, na hora em que a lista `fases` nasceu.
3. Na próxima vez que o jogador abrir a mochila, ela aparece na lista, pronta pra usar.

### E SE O JOGADOR NÃO USAR O ITEM ALI?

Sem problema. O item fica guardado no `Inventario` até ser usado — inclusive numa fase seguinte, contra um inimigo diferente do que o entregou. A mochila não sabe nem se importa de onde cada item veio.

FIM DA AULA DE HOJE

# Três itens, um padrão, zero if novo

Na próxima aula: o personagem melhora depois de cada vitória — o começo da progressão.

- ✓ PocaDeForca existe e aumenta o ataque de quem usa
- ✓ AnelDeProtecao existe e aumenta a defesa de quem usa
- ✓ Os dois já são prêmio de pelo menos uma fase
- ✓ Nenhum arquivo além de `itens.py` e a lista `fases` precisou mudar

## PRÉVIA DA PRÓXIMA AULA

```
def melhorar_ataque(self, quantidade):  
    self.ataque += quantidade
```